Natural Language Processing - Lecture 7: Morphological processing (5 Nov)

Session context

The 61-page deck is titled Natural Language Processing: Morphological processing and names Miryam de Lhoneux. The recording discusses morphology, its processing tasks, finite-state methods, neural models, and tokenization. It begins with lemmatization and tokenization on slides 7-8; the first six slides are not discussed.

Lecture overview

Lecture 7 introduces morphology as the study of word internal structure, centering on the morpheme as the smallest unit of meaning. Morphemes are categorized into stems (or lemmas), which provide core meaning, and affixes, which add grammatical or semantic information; these affixes are further distinguished as bound (unable to stand alone) or free. The lecture contrasts inflectional morphology, which creates grammatical distinctions like tense or number without changing the part of speech, against derivational morphology, which alters word meaning and often shifts the part of speech. This distinction is critical for NLP tasks such as coreference resolution, machine translation, and lemmatization. Furthermore, languages exhibit varying morphological typologies ranging from isolating to polysynthetic, utilizing either concatenative mechanisms (adding affixes) or non-concatenative processes like infixation, circumfixation, and reduplication.

The second half of the lecture focuses on data-driven approaches to morphological processing and tokenization. Morphological analysis is framed as a word-level tagging problem where models predict lemmas and features simultaneously, often employing multitask architectures that outperform sequential methods by avoiding invalid feature combinations. While classification-based models use transformer outputs for tag prediction, encoder-decoder architectures leverage character-level information to handle inflection and reduce sparsity. In subword tokenization, standard Byte Pair Encoding (BPE) is critiqued for creating splits that violate morphological boundaries based solely on frequency. To address this, the lecture discusses morphologically-informed strategies, including BPE-dropout to introduce segmentation variability and BPE-knockout, which uses a handcrafted lexicon to specifically disable merges that cross morpheme boundaries, thereby improving compositional generalization and downstream performance.

Slide 1: Natural Language Processing - Morphological processing

Title page for course H02B1A, presented by Miryam de Lhoneux. It identifies the subject and speaker; it does not introduce a specific method.

Slide 2: Introduction outline

The lecture roadmap moves from morphology and its processing tasks to rule-based methods, data-driven methods, morphology-informed subword tokenization, and a summary. The recording starts after this opening material.

Slide 3: What is morphology?

Morphology studies the internal structure of words. The branching diagram uses independently to preview how a complex word can be built from smaller meaningful pieces.

Slide 4: Morphemes

A morpheme is a smallest meaningful unit in the analysis used here. The diagram decomposes independently into a stem and affixes; the stem supplies the core lexical content, while affixes add grammatical or derivational information.

Slide 5: Why morphology helps understanding

Number marking can support coreference. In “The dogs chase the cat and they fall,” the plural subject dogs is compatible with plural they, whereas cat is singular. Agreement is a useful clue, though context and discourse can still affect pronoun interpretation.

Slide 6: Why morphology helps generation

Morphological features can also guide generation. A translation system should preserve the plural feature of English dogs when producing French les chiens. The slide illustrates why inflectional information matters in translation.

Slide 7: Indirect use - lemmatization for retrieval

Lemmatization maps inflected forms to a lemma so a search can match related forms. In the illustration, query buy can match documents containing forms such as bought after normalization. This is an example of morphology improving retrieval, not a guarantee that every search system should lemmatize every query.

Slide 8: Indirect use - subword tokenization

The slide contrasts a frequency-based split of nonsensical (nons-ens-ical) with a morphology-aware split (non-sens-ical). BPE-style methods merge frequent adjacent symbols; without linguistic constraints, a learned boundary can cross a morpheme boundary. The displayed GPT-4o tokenization is an example to inspect, not a universal tokenization rule.

Slide 9: Morphology outline

This agenda build highlights morphology after the introductory section. It introduces the next set of definitions rather than a new technical claim.

Slide 10: Morphemes, stems, and affixes

Stems carry core lexical content and affixes add information. Bound morphemes such as plural -s, -ly, and -ed cannot stand alone; free morphemes such as dog and walk can. The lecturer groups stem, lemma, base form, root, and lexeme together for this course, while noting that linguists distinguish these terms in more precise analyses.

Slide 11: Inflectional morphemes

Inflection changes a word form to express grammatical distinctions while usually preserving its part of speech and core lexical identity: dog → dogs, eat → eats, and eat → eating. The wording “does not change meaning” is a useful simplification; inflection adds grammatical meaning such as number, tense, or aspect.

Slide 12: Derivational morphemes

Derivation forms a new lexeme and often changes part of speech or lexical meaning: appoint + -ee → appointee, help + -less → helpless, and strength + -en → strengthen. A part-of-speech change is common but not required for every derivational process.

Slide 13: Concatenative morphology

Concatenative morphology forms words by attaching affixes in sequence. The slide shows prefixes such as dis-, suffixes such as -ment, and their combination in disestablishment.

Slide 14: Non-concatenative morphology

The examples show three ways morphology can alter a word beyond simply adding an affix at one edge: Tagalog infixation (hingi → humingi), the German participial pattern ge-…-t (sagen → gesagt), and Motu reduplication (mahuta → mahutamahuta). These illustrate different language-specific processes; they are not all the same kind of operation.

Slide 15: Morphological typology - English, Chinese, Russian

The parallel examples illustrate differences in how grammatical information is expressed. The Russian example packages more grammatical material within a word than the Chinese example; English lies between these examples in this illustration. “Analytic” and “synthetic” describe tendencies, not a single fixed scale that fully classifies every construction in a language.

Slide 16: Morphemes per word

The diagram places languages along an approximate continuum of morphemes per word, based on Greenberg’s 1960 quantitative typology and reproduced in Jurafsky and Martin. Such counts depend on the sample and analysis; the figure is a historical comparison, not a timeless ranking of whole languages.

Slide 17: West Greenlandic example

The long West Greenlandic word is used to illustrate polysynthesis: one word can encode material that English expresses with a phrase, including the idea “really good for catching polar bears with.” The gloss shows why morphology-rich forms can require explicit segmentation and feature analysis.

Slide 18: Agglutinative and fusional patterns

Agglutinative morphology tends to use identifiable morphemes with relatively separable functions; fusional morphology can bundle several grammatical features into one form. Turkish and Russian are examples, respectively, but the distinction is a continuum and neither label describes every word in a language.

Slide 19: Morphological complexity and NLP

The slide summarizes research on whether morphologically complex languages are harder for NLP models and how performance gaps might be addressed. Treat “agglutinative and fusional languages are more complex” as a broad teaching generalization, not a universal fact: morphological complexity has several dimensions, and language-model comparisons can be confounded by data, tokenization, and evaluation choices. Park et al. (2021) find associations in specific experiments; Poelman et al. (2025) emphasize controls needed for reliable comparisons.

Slide 20: Processing tasks outline

This agenda build highlights the processing-task section: analysis and generation, followed by rule-based and data-driven approaches. It is a transition page.

Slide 21: Morphological analysis

For puppies, lemmatization returns puppy, segmentation represents the plural suffix, and tagging records lemma, part of speech, and number. The slide writes the segmentation as puppy|s; spelling changes y to i in the surface form, so this notation is a morphological analysis rather than a literal character split.

Slide 22: Analysis in isolation - saw

Without context, saw has multiple analyses: past tense of see, present-tense saw (“I saw wood,” from the verb meaning cut with a saw), or the noun naming the tool. Context selects among these candidate lemma/POS/feature bundles. The second analysis is the present form of a different verb, saw, not a present-tense form of see.

Slide 23: Analysis in context - cooler

Context disambiguates cooler: in “The cooler was full of orange juice” it is a singular noun; in “The daisy prefers cooler weather” it is the comparative adjective with lemma cool. A morphology analyzer can use sentence context to choose the appropriate analysis.

Slide 24: Inflection generation

Given a lemma and a feature bundle, a generator produces a surface form: permaban with past-participle features becomes permabanned; German Stiefvater with plural accusative features becomes Stiefväter; Hungarian homeosztázis with dative plural features becomes homeosztázisoknak.

Slide 25: Paradigm generation

Paradigm generation produces a lemma’s set of inflected forms. The German verb schleichen is shown with person/number/present forms and the participle geschlichen. The ellipsis indicates the list continues; it is not a claim that the displayed forms exhaust the paradigm.

Slide 26: Generation in context

The schematic example supplies lemma dog and a plural context (are barking) and asks for the agreeing form dogs. This is context-guided inflection generation; it does not mean a morphology generator alone repairs the whole ungrammatical sentence.

Slide 27: Rule-based processing outline

This agenda build marks the transition to rule-based morphology.

Slide 28: English noun inflection

Regular plural patterns include cat → cats and thrush → thrushes. Irregular forms such as mouse → mice and ox → oxen require lexical entries or explicit exceptions rather than the general -s rule.

Slide 29: Finite-state automaton for nouns

An FSA recognizes whether a string follows an encoded pattern. The diagram has a path for regular noun stems followed by plural -s, plus paths for listed irregular singular and plural forms. An automaton accepts or rejects strings; a transducer is needed when the system must map between a lemma/feature representation and a surface form.

Slide 30: Lexicon requirements

The noun-inflexion example needs a list of regular stems and entries for irregular singular/plural pairs. A productive rule can apply to a new stem once the lexicon includes it; irregular or newly added vocabulary may require updates. This is more precise than saying every new word always requires a new rule.

Slide 31: English verbal inflection

The FSA separates regular verb stems such as walk, fry, and talk from irregular stems such as catch, speak, and sing. Regular paths add forms such as past -ed, participial -ing, or third-person singular -s; irregular paths list forms such as caught, spoke, and sang. The spoken lecture then distinguishes an FST, which can generate a form such as walk + past → walked, from an FSA that merely recognizes forms.

Slide 32: Finite-state diagram for derivation

The slide caption says “A FSA for English nominal inflection,” while the diagram illustrates derivational paths across adjectives, verbs, nouns, and adverbs. It includes global → globalize → globalization and helpful → helpfully/helpfulness. These examples show derivational word formation, not just noun inflection.

Slide 33: Using FSAs and FSTs

The exercise shows that this particular automaton accepts helpfulness and helpfully but rejects helpfulization. An FSA can recognize forms licensed by its paths; an FST can additionally map forms to analyses or generate them from lemmas and features. These decisions depend on the lexicon and rules encoded in the machine.

Slide 34: Limits of FSAs

Finite-state systems can require substantial lexical and linguistic work, and the vocabulary may need maintenance. A finite-state machine cannot model unrestricted copying of an arbitrarily long sequence (the reduplication case on the slide), although restricted or bounded reduplication patterns can be represented. Rule-based transducers remain useful; they are not universally obsolete.

Slide 35: Data-driven processing outline

This agenda build highlights data-driven methods after the rule-based section.

Slide 36: Morphological analysis as tagging

The French sentence illustrates word-level predictions of lemmas and feature bundles alongside surface forms. A tagger can predict a word’s lemma, UPOS, and features such as gender, number, tense, person, and mood; the exact features vary by word and language.

Slide 37: Multitask architecture

The UDify-style diagram shares a BERT representation across heads for dependency label/head, lemma, morphological features, and UPOS. The sentence example (“The best optimizer is grad student descent”) illustrates shared contextual representations feeding several outputs. Kondratyuk and Straka (2019) report one multilingual model trained across 75 languages and 124 Universal Dependencies treebanks; the picture is a multitask architecture, not a dedicated morphology-only model.

Slide 38: Tag prediction

Each input word is represented by a transformer and passed to a feed-forward network that predicts tags. Because subword tokenization can split a word such as optimizer into several pieces, the model must form one word representation from their vectors. Mean pooling is one option; the lecturer notes that using the first subword's output often works well because each contextualized subword already carries information from the full context. A softmax scores candidate tags and argmax selects the predicted tag. The same representation can support POS, morphological, lemma, and dependency predictions.

Slide 39: Joint versus split feature tags

One approach predicts a single combined feature bundle; another predicts each feature separately. Separate heads can reduce sparsity, while a combined label can capture dependencies and avoid invalid combinations. The lecturer reports that joint all-in-one labels work best in the cited setting; this is an empirical result, not a universal rule for every data set or architecture.

Slide 40: Encoder-decoder alternative

Some morphology tasks can generate an output sequence with an encoder-decoder instead of predicting one tag from an encoder representation. This is especially suitable when the output is a lemma or inflected word string.

Slide 41: Sequence-processing architectures

The diagram contrasts decoder-only autoregressive processing, encoder-only contextual representations, and encoder-decoder sequence generation. The arrows show how inputs and outputs are connected; see the earlier neural-architectures lecture for the detailed theory.

Slide 42: Encoder-decoder lemmatization

The example encodes the characters of an inflected word together with POS and morphological features, then generates lemma characters autoregressively. Character-level input lets the model use the spelling overlap between forms such as had and have, rather than selecting a lemma only from a flat label set. Kanerva et al. describe this sequence-to-sequence approach with morphosyntactic context.

Slide 43: Sequence-to-sequence generation

For morphological generation, the encoder-decoder receives a lemma and feature specification and generates the requested surface form. It is the same sequence-to-sequence principle as machine translation, applied at the word level.

Slide 44: Inflection generation example

The diagram conditions a character-level decoder on the verb dun and a simple-past feature, producing dunned. This illustrates character-by-character reinflection; it does not demonstrate that seq2seq is best for every language or resource level.

Slide 45: Evaluation and reported performance

Exact word-level accuracy is common for lemmatization and morphological tagging, though the right metric depends on the task. The slide gives a broad 50-95% range for reported morphological-analysis results when data is sufficient. Read this as a lecture snapshot across languages and data sets; it is not a single current benchmark or guarantee.

Slide 46: Subword-tokenization outline

This agenda build moves to morphology-informed subword tokenization.

Slide 47: BPE merge example

The corpus lists word types with frequencies, starts from characters, and merges frequent adjacent pairs. The example first merges e+r → er, then er+_ → er_, where _ marks a word boundary. BPE follows pair frequency; it does not know that er happens to be a suffix in some words.

Slide 48: BPE merge tree

Each internal node records a learned merge and each leaf is a character/symbol. The tree for newer_ shows how earlier merges (e+r, er+_, n+e, ne+w) combine into larger pieces. The tree records merge history; it is not a parse tree from linguistic syntax.

Slide 49: Imagined morphology-aligned tree

The hypothetical tree for independently groups in-, dependent, and -ly so each piece stays within one morpheme. It is an idealized example, not the output of ordinary frequency-based BPE.

Slide 50: Actual RoBERTa BPE tree

The actual tree for independently contains a merge joining the prefix in with the following d, crossing the intended in- | dependent boundary. Ġ is RoBERTa's word-start/leading-space marker. Comparing this tree with slide 49 shows why frequency-based merges need not follow morphology.

Slide 51: BPE tree for horseshoe

The displayed tree groups the beginning of shoe with horse, producing a piece like horses plus hoe rather than the intuitive compound split horse | shoe. It is a concrete example where a token boundary can cross a meaningful compound boundary and change how the word's parts are represented.

Slide 52: Deterministic segmentation and composition

Ordinary BPE assigns each word a deterministic segmentation. A suffix such as -ly may exist in the vocabulary yet remain underused when frequent adverbs are stored as whole tokens. That can make rare forms such as inherently harder to represent.

Slide 53: BPE-dropout

BPE-dropout randomly disables some merges during training, allowing the same word to appear with different segmentations. Provilkov et al. found translation improvements in their experiments when dropout is used for training and standard BPE at inference. Randomly disabling merges can also remove good merges; the method is subword regularization, not a morphology-aware rule system.

Slide 54: BPE-knockout

BPE-knockout identifies and removes merges that cross morphological boundaries using morphological supervision, rather than disabling merges at random. For independently, merges that fuse material from separate morphemes are pruned. The 2024 paper reports better boundary alignment and improvements on particular token-based tasks; do not generalize that outcome to every model or language.

Slide 55: BPE-knockout tree

For independently, TkTkT's morphology-aligned BPE-knockout tree groups in-, dependent, and -ly; dependent is also shown as an internal constituent.

Slide 56: BPE-knockout conclusion

The slide concludes that morphological information can guide subword segmentation and improve downstream performance in the reported experiments. The approach requires a handcrafted lexicon and leaves room for further work.

Slide 57: Summary outline

This agenda build highlights the final summary section.

Slide 58: Summary

The recap covers morphology, analysis and generation tasks, finite-state rule-based methods, neural approaches, and morphology-informed subword tokenization. It is a list of topics rather than a new definition.

Slide 59: Outlook - direct morphology and English

The slide suggests that explicit morphology is less necessary for English because its inflection is relatively limited and training data is abundant. The spoken lecture itself qualifies the point. “Direct use is no longer relevant for English” is too strong: morphology can still help with generation, low-resource data, structured analysis, retrieval, and particular tokenization problems. Language-modeling results depend on language coverage, data, tokenization, and evaluation; Park et al. (2021) and Poelman et al. (2025) discuss these experimental limits.

Slide 60: Acknowledgments

The course credits Jimmy Lin for inspiration for the morphology slides and Kyle Gorman for the processing-task slides. This is attribution, not a technical claim.

Slide 61: References

The bibliography includes work on typology, finite-state morphology, neural lemmatization and reinflection, multilingual parsing, and morphology-aware tokenization. Key papers used to check the explanations include Kondratyuk and Straka (2019), Kanerva et al. (2021), Senuma and Aizawa (2017), BPE-dropout (2020), BPE-knockout (2024), Park et al. (2021), and Poelman et al. (2025).